Read Connected Data

Now that we have some data in both tables, let's select the data that is connected together.

The `team` table has this data:

id	name	headquarters
1	Preventers	Sharp Tower
2	Z-Force	Sister Margaret's Bar

And the `hero` table has this data:

id	name	secret_name	age	team_id
1	Deadpond	Dive Wilson	null	2
2	Rusty-Man	Tommy Sharp	48	1
3	Spider-Boy	Pedro Parqueador	null	null

We will continue with the code in the previous example and we will add more things to it.

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

`SELECT` Connected Data with SQL

Let's start seeing how SQL works when selecting connected data. This is where SQL databases actually shine.

If you don't have a `database.db` file, run that previous program we had written (or copy it from the preview above) to create it.

Now open **DB Browser for SQLite** and open the `database.db` file.

To `SELECT` connected data we use the same keywords we have used before, but now we combine the two tables.

Let's get each hero with the `id`, `name`, and the team `name`:

```
SELECT hero.id, hero.name, team.name
FROM hero, team
WHERE hero.team_id = team.id
```

Info

Because we have two columns called `name`, one for `hero` and one for `team`, we can specify them with the prefix of the table name and the dot to make it explicit what we refer to.

Notice that now in the `WHERE` part we are not comparing one column with a literal value (like `hero.name = "Deadpond"`), but we are comparing two columns.

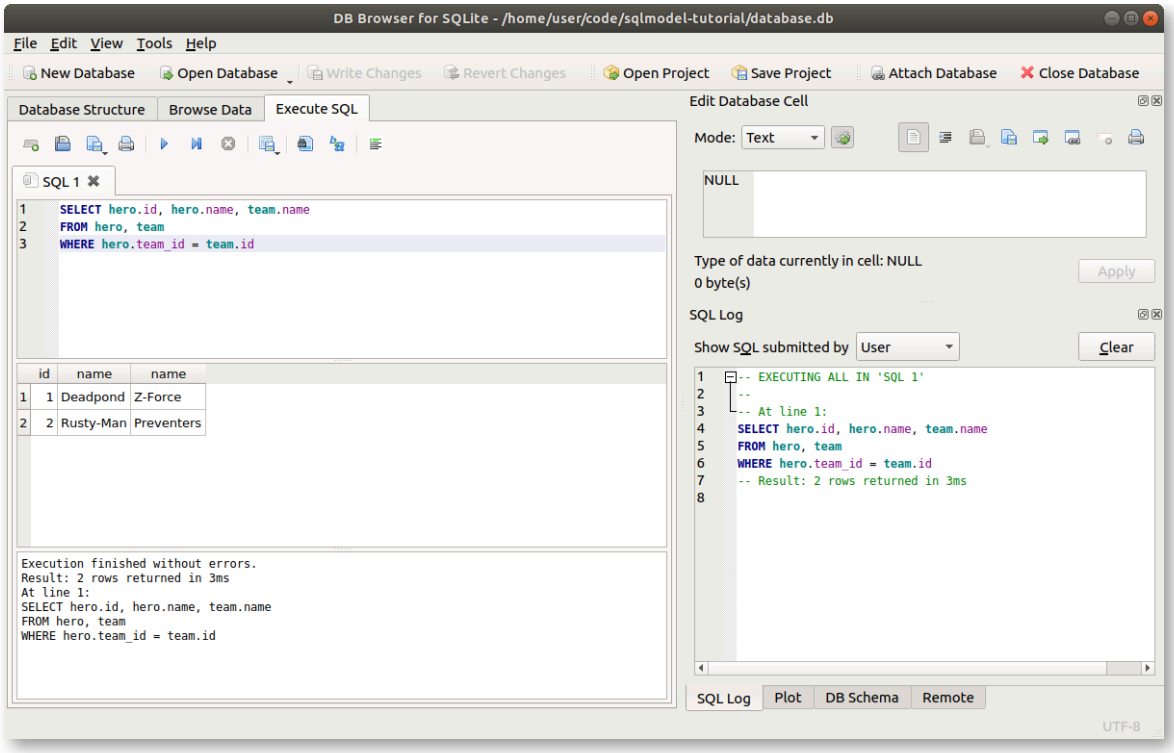
It means, more or less:

> Hey SQL database 👋, please go and `SELECT` some data for me.
>
> I'll first tell you the columns I want:
>
> - `id` of the `hero` table
> - `name` of the `hero` table
> - `name` of the `team` table
>
> I want you to get that data `FROM` the tables `hero` and `team`.
>
> And I don't want you to combine each hero with each possible team. Instead, for each hero, go and check each possible team, but give me only the ones `WHERE` the `hero.team_id` is the same as the `team.id`.

If we execute that SQL, it will return the table:

id	name	name
1	Deadpond	Z-Force
2	Rusty-Man	Preventers

You can go ahead and try it in **DB Browser for SQLite**:



Note

Wait, what about Spider-Boy? 😱

He doesn't have a team, so his `team_id` is `NULL` in the database. And this SQL is comparing that `NULL` from the `team_id` with all the `id` fields in the rows in the `team` table.

As there's no team with an ID of `NULL`, it doesn't find a match.

But we'll see how to fix that later with a `LEFT JOIN`.

Select Related Data with **SQLModel**

Now let's use SQLModel to do the same select.

We'll create a function `select_heroes()` just as we did before, but now we'll work with two tables.

Remember SQLModel's `select()` function? It can take more than one argument.

So, we can pass the `Hero` and `Team` model classes. And we can also use both their columns in the `.where()` part:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).where(Hero.team_id == Team.id)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).where(Hero.team_id == Team.id)
        results = session.exec(statement)
        for hero, team in results:
            print("Hero:", hero, "Team:", team)


def main():
    create_db_and_tables()
```

```
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Notice that in the comparison with `==` we are using the class attributes for both `Hero.team_id` and `Team.id`.

That will generate the appropriate **expression** object that will be converted to the right SQL, equivalent to the SQL example we saw above.

Now we can execute it and get the `results` object.

And as we used `select` with two models, we will receive tuples of instances of those two models, so we can iterate over them naturally in a `for` loop:

Python 3.10+

```
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).where(Hero.team_id == Team.id)
        results = session.exec(statement)
        for hero, team in results:
            print("Hero:", hero, "Team:", team)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).where(Hero.team_id == Team.id)
        results = session.exec(statement)
        for hero, team in results:
            print("Hero:", hero, "Team:", team)


def main():
    create_db_and_tables()
```

```
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

For each iteration in the `for` loop we get a tuple with an instance of the class `Hero` and an instance of the class `Team`.

And in this `for` loop we assign them to the variable `hero` and the variable `team`.

Info

There was a lot of research, design, and work behind **SQLModel** to make this provide the best possible developer experience.

And you should get autocompletion and inline errors in your editor for both `hero` and `team`. 🎉

Add It to Main

As always, we must remember to add this new `select_heroes()` function to the `main()` function to make sure it is executed when we call this program from the command line.

Python 3.10+

```
# Code above omitted 👆

def main():
    create_db_and_tables()
    create_heroes()
    select_heroes()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).where(Hero.team_id == Team.id)
        results = session.exec(statement)
        for hero, team in results:
            print("Hero:", hero, "Team:", team)


def main():
    create_db_and_tables()
```

```
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Run the Program

Now we can run the program and see how it shows us each hero with their corresponding team:

```bash
```

`JOIN` Tables with SQL

There's an alternative syntax for that SQL query from above using the keyword `JOIN` instead of `WHERE`.

This is the same version from above, using `WHERE`:

```
SELECT hero.id, hero.name, team.name
FROM hero, team
WHERE hero.team_id = team.id
```

And this is the alternative version using `JOIN`:

```
SELECT hero.id, hero.name, team.name
FROM hero
JOIN team
ON hero.team_id = team.id
```

Both are equivalent. The differences in the SQL code are that instead of passing the `team` to the `FROM` part (also called `FROM` clause) we add a `JOIN` and put the `team` table there.

And then, instead of putting a `WHERE` with a condition, we put an `ON` keyword with the condition, because `ON` is the one that comes with `JOIN`. 💁

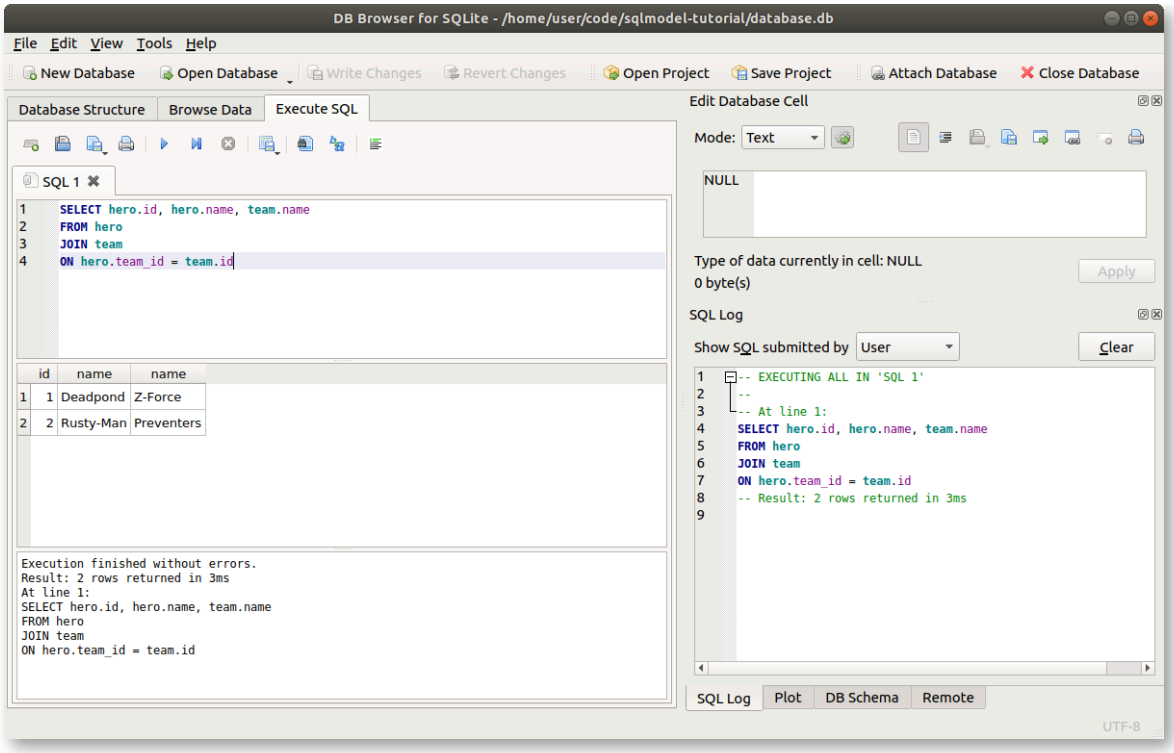
So, this second version means, more or less:

> Hey SQL database 👋, please go and `SELECT` some data for me.
>
> I'll first tell you the columns I want:
>
> - `id` of the `hero` table
> - `name` of the `hero` table
> - `name` of the `team` table
>
> ...up to here it's the same as before, LOL.
>
> Now, I want you to get that data starting `FROM` the table `hero`.
>
> And to get the rest of the data, I want you to `JOIN` it with the table `team`.
>
> And I want you to join those two tables `ON` the combinations of rows that have the `hero.team_id` with the same value as the `team.id`.
>
> Did I say all this before already? I feel like I'm just repeating myself. 🤔

That will return the same table as before:

id	name	name
1	Deadpond	Z-Force
2	Rusty-Man	Preventers

Also in **DB Browser for SQLite**:



Tip

Why bother with all this if the result is the same?

This `JOIN` will be useful in a bit to be able to also get Spider-Boy, even if he doesn't have a team.

Join Tables in **SQLModel**

The same way there's a `.where()` available when using `select()`, there's also a `.join()`.

And in SQLModel (actually SQLAlchemy), when using the `.join()`, because we already declared what is the `foreign_key` when creating the models, we don't have to pass an `ON` part, it is inferred automatically:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).join(Team)
        results = session.exec(statement)
        for hero, team in results:
            print("Hero:", hero, "Team:", team)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).join(Team)
        results = session.exec(statement)
        for hero, team in results:
            print("Hero:", hero, "Team:", team)


def main():
    create_db_and_tables()
```

```
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Also notice that we are still including `Team` in the `select(Hero, Team)`, because we still want to access that data.

This is equivalent to the previous example.

And if we run it in the command line, it will output:

```bash
```

`JOIN` Tables with SQL and `LEFT OUTER` (Maybe `JOIN`)

When working with a `JOIN`, you can imagine that you start with a table on the `FROM` part and put that table in an imaginary space on the **left** side.

And then you want another table to `JOIN` the result.

And you put that second table in the **right** side on that imaginary space.

And then you tell the database `ON` which condition it should join those two tables and give you the results back.

But by default, only the rows from both left and right that match the condition will be returned.

hero				
id	name	secret_name	age	team_id
1	Deadpond	Dive Wilson	null	2
2	Spider-Boy	Pedro Parqueador	null	1
3	Rusty-Man	Tommy Sharp	48	1

team		
id	name	headquarters
1	Preventers	Sharp Tower
2	Z-Force	Sister Margaret's Bar

In this example of tables above 👆, it would return all the heroes, because every hero has a `team_id`, so every hero can be joined with the `team` table:

id	name	name
1	Deadpond	Z-Force
2	Rusty-Man	Preventers
3	Spider-Boy	Preventers

Foreign Keys with `NULL`

But in the database that we are working with in the code above, **Spider-Boy** doesn't have any team, the value of `team_id` is `NULL` in the database.

So there's no way to join the **Spider-Boy** row with some row in the `team` table:

hero				
id	name	secret_name	age	team_id
1	Deadpond	Dive Wilson	null	2
2	Spider-Boy	Pedro Parqueador	null	null
3	Rusty-Man	Tommy Sharp	48	1

team		
id	name	headquarters
1	Preventers	Sharp Tower
2	Z-Force	Sister Margaret's Bar

Running the same SQL we used above, the resulting table would not include **Spider-Boy** 😱:

id	name	name
1	Deadpond	Z-Force
2	Rusty-Man	Preventers

Include Everything on the `LEFT OUTER`

In this case, that we want to include all heroes in the result even if they don't have a team, we can extend that same SQL using a `JOIN` from above and add a `LEFT OUTER` right before `JOIN`:

```
SELECT hero.id, hero.name, team.name
FROM hero
LEFT OUTER JOIN team
ON hero.team_id = team.id
```

This `LEFT OUTER` part tells the database that we want to keep everything on the first table, the one on the `LEFT` in the imaginary space, even if those rows would be left **out**, so we want it to include the `OUTER` rows too. In this case, every hero with or without a team.

And that would return the following result, including **Spider-Boy** 🎉:

id	name	name
1	Deadpond	Z-Force
2	Rusty-Man	Preventers
3	Spider-Boy	null

Tip

The only difference between this query and the previous is that extra `LEFT OUTER`.

And here's another of the SQL variations, you could write `LEFT OUTER JOIN` or just `LEFT JOIN`, it means the same.

Join Tables in **SQLModel** with `LEFT OUTER`

Now let's replicate the same query in **SQLModel**.

`.join()` has a parameter we can use `isouter=True` to make the `JOIN` be a `LEFT OUTER JOIN`:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).join(Team, isouter=True)
        results = session.exec(statement)
        for hero, team in results:
            print("Hero:", hero, "Team:", team)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).join(Team, isouter=True)
        results = session.exec(statement)
        for hero, team in results:
            print("Hero:", hero, "Team:", team)


def main():
    create_db_and_tables()
```

```
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

And if we run it, it will output:

```bash
```

What Goes in `select()`

You might be wondering why we put the `Team` in the `select()` and not just in the `.join()`.

And then why we didn't include `Hero` in the `.join()`. 🤔

In SQLModel (actually in SQLAlchemy), all these functions and tools try to **replicate** how it would be to work with the **SQL** language.

Remember that `SELECT` defines the columns to get and `WHERE` how to filter them?.

This also applies here, but with `JOIN` and `ON`.

Select Only Heroes But Join with Teams

If we only put the `Team` in the `.join()` and not in the `select()` function, we would not get the `team` data.

But we would still be able to **filter** the rows with it. 🤓

We could even add some additional `.where()` after `.join()` to filter the data more, for example to return only the heroes from one team:

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).join(Team).where(Team.name == "Preventers")
        results = session.exec(statement)
        for hero in results:
            print("Preventer Hero:", hero)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero).join(Team).where(Team.name == "Preventers")
        results = session.exec(statement)
        for hero in results:
            print("Preventer Hero:", hero)


def main():
    create_db_and_tables()
```

```
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

Here we are **filtering** with `.where()` to get only the heroes that belong to the **Preventers** team.

But we are still only requesting the data from the heroes, not their teams.

If we run that, it would output:

```bash
```

Include the `Team`

By putting the `Team` in `select()` we tell **SQLModel** and the database that we want the team data too.

Python 3.10+

```python
# Code above omitted 👆

def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).join(Team).where(Team.name == "Preventers")
        results = session.exec(statement)
        for hero, team in results:
            print("Preventer Hero:", hero, "Team:", team)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Session, SQLModel, create_engine, select


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    team_id: int | None = Field(default=None, foreign_key="team.id")


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")
        session.add(team_preventers)
        session.add(team_z_force)
        session.commit()

        hero_deadpond = Hero(
            name="Deadpond", secret_name="Dive Wilson", team_id=team_z_force.id
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            team_id=team_preventers.id,
        )
        hero_spider_boy = Hero(name="Spider-Boy", secret_name="Pedro Parqueador")
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Created hero:", hero_deadpond)
        print("Created hero:", hero_rusty_man)
        print("Created hero:", hero_spider_boy)


def select_heroes():
    with Session(engine) as session:
        statement = select(Hero, Team).join(Team).where(Team.name == "Preventers")
        results = session.exec(statement)
        for hero, team in results:
            print("Preventer Hero:", hero, "Team:", team)


def main():
    create_db_and_tables()
```

```
    create_heroes()
    select_heroes()


if __name__ == "__main__":
    main()
```

And if we run that, it will output:

```bash
```

We still have to `.join()` because otherwise it would just compute all the possible combinations of heroes and teams, for example including **Rusty-Man** with **Preventers** and also **Rusty-Man** with **Z-Force**, which would be a mistake.

Relationship Attributes

Here we have been using the pure class models directly, but in a future chapter we will also see how to use **Relationship Attributes** that let us interact with the database in a way much more close to the code with Python objects.

And we will also see how to load their data in a different, simpler way, achieving the same we achieved here. ✨